БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики и информатики
Кафедра технологий программирования

Лабораторная работа 2

Коммуникация в многоконтейнерных системах

Дисциплина «Непрерывное интегрирование и сборка программного обеспечения»

Выполнил: Атаев Перман
Группа 13
Вариант 1
Минск 2026

Я реализовал передачу данных между двумя контейнерами через файлы и выполнил три примера обмена сообщениями с RabbitMQ. В первом задании получен результат 63. В упражнении 2.2 проверены отправка и получение сообщений, распределение задач, повторная доставка и рассылка подписчикам.

Объём для группы 13: задание 1, упражнение 2.2 и все 30 контрольных вопросов. Публикация на GitHub и Pull Request оформляются отдельно после подготовки отчёта; в представленных материалах они пока не подтверждены.

1 Цель и подготовка окружения

Цель работы — освоить коммуникацию компонентов распределённой системы через общие файлы и брокер сообщений. Я использовал Windows PowerShell, Docker Desktop в режиме Linux-контейнеров и Python 3.14.

Проверка показала: Docker 29.8.1, Docker Compose v5.5.1, Git 2.56.0.windows.1 и Python 3.14.7. Для проекта я создал каталог C:\Users\perma\Desktop\lab2\lab2nispo.

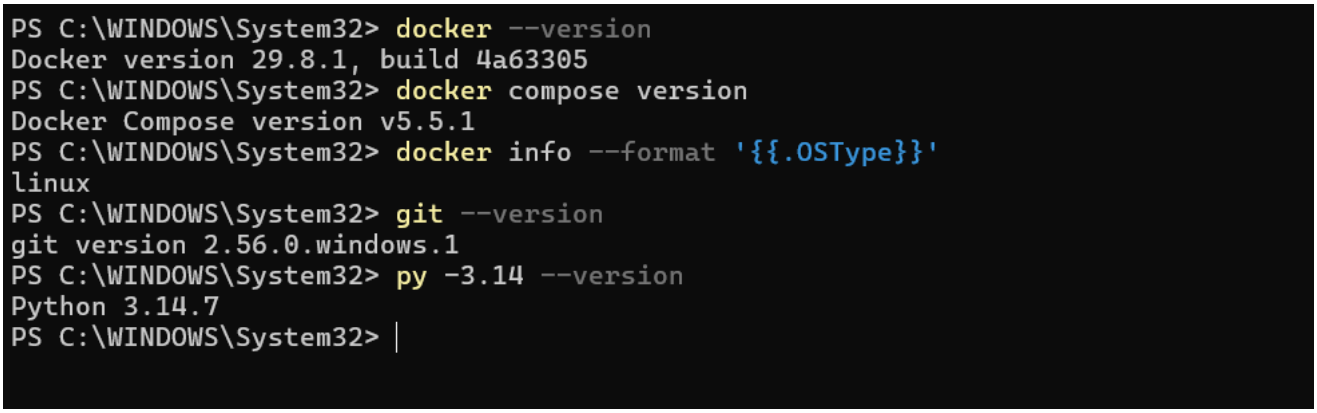


Рисунок 1. Проверка версий и режима Linux

Я инициализировал локальный Git-репозиторий с веткой main, создал начальный README с именем, группой и вариантом и сохранил его коммитом be7d8e5. Затем создал ветку task1.

```
git init -b main
git add README.md
git commit -m "Initialize lab 2"
git switch -c task1
```

2 Задание 1 и исходные данные

По варианту 1 первая программа должна найти самый большой по объёму файл и поместить его содержимое в /var/result/data.txt. Вторая программа вычисляет произведение первого и последнего числа, записывает его в /var/result/result.txt и выводит на экран.

Я создал каталоги data, result-1, result, worker1 и worker2 внутри task1. Для воспроизводимости исходный файл копируется, а не удаляется. При равных размерах программа выбирает первый файл по имени; служебный .gitkeep исключён из поиска.

В data находятся small.txt с числами 2, 3, 4; medium.txt с числами 10, 20, 30, 40; large.txt с числами 7, 100, 200, 300, 400, 500, 600, 9. При проверке на Windows их размеры составили 7, 13 и 29 байт соответственно. Размеры могут измениться при преобразовании окончаний строк Git, но порядок размеров этих файлов сохранится.



Рисунок 5. Размеры исходных файлов и содержимое large.txt

Ожидаемый результат: 7 × 9 = 63. Этот ответ я использовал для проверки обеих программ.

3 Локальная проверка программ

В worker1/app.py я использовал pathlib для поиска файлов и shutil.copyfile для копирования выбранного файла. Аргументы --input-dir и --output-dir позволяют проверить программу в Windows; в контейнере используются стандартные пути /var/data и /var/result.

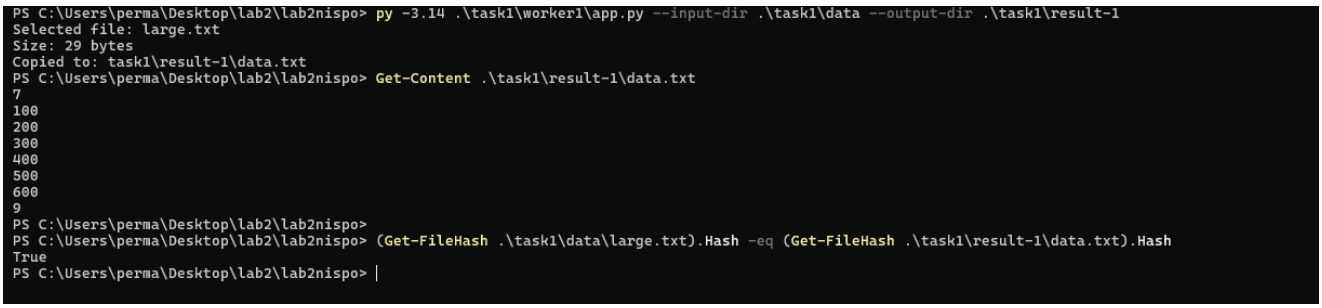


Рисунок 7. Выбор large.txt и совпадение хешей исходного файла и копии

Во второй программе я читаю data.txt в кодировке utf-8-sig, разбиваю содержимое по пробельным символам и преобразую значения в целые числа. Пустой файл или некорректное число приводят к сообщению об ошибке и коду завершения 1. На подготовленных данных программа вывела 63 и записала его в result.txt.

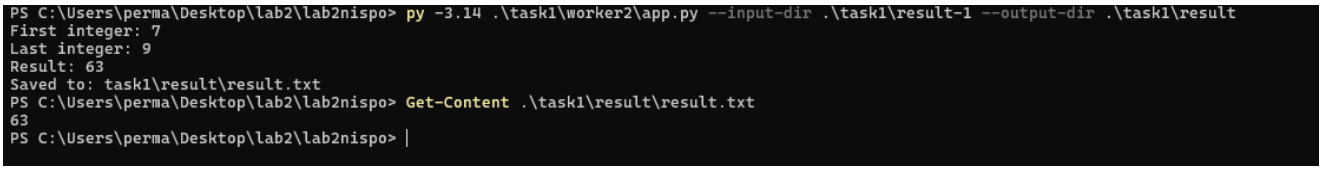


Рисунок 9. Локальное вычисление произведения и проверка result.txt

Проверка хешей вернула True: промежуточный файл точно совпал с выбранным исходным файлом. Отдельных скриншотов проверки пустого файла и неверного числа нет; эти случаи предусмотрены кодом, но не включены в список выполненных тестов.

4 Контейнеризация и передача файлов

Для обеих программ я создал Dockerfile на базе python:3.14-slim. Дополнительные зависимости не нужны: используются модули стандартной библиотеки. В Dockerfile указаны рабочий каталог /app, копирование app.py и команда запуска Python. Блоки инструкций снабжены комментариями.

В Compose я определил два сервиса с отдельными контекстами сборки worker1 и worker2. Каталог ./data монтируется в /var/data первого контейнера только для чтения. Каталог ./result-1 монтируется в /var/result первого контейнера и в /var/data второго только для чтения. Каталог ./result монтируется в /var/result второго контейнера.

Зависимость service_completed_successfully разрешает запуск worker2 после успешного завершения worker1. Поэтому второй процесс читает уже подготовленный промежуточный файл. Перед запуском я удалил результаты локальной проверки, чтобы убедиться в создании новых файлов контейнерами.

```
docker compose config -q
docker compose up --build
docker compose ps -a
Get-Content .\result-1\data.txt
Get-Content .\result\result.txt
```

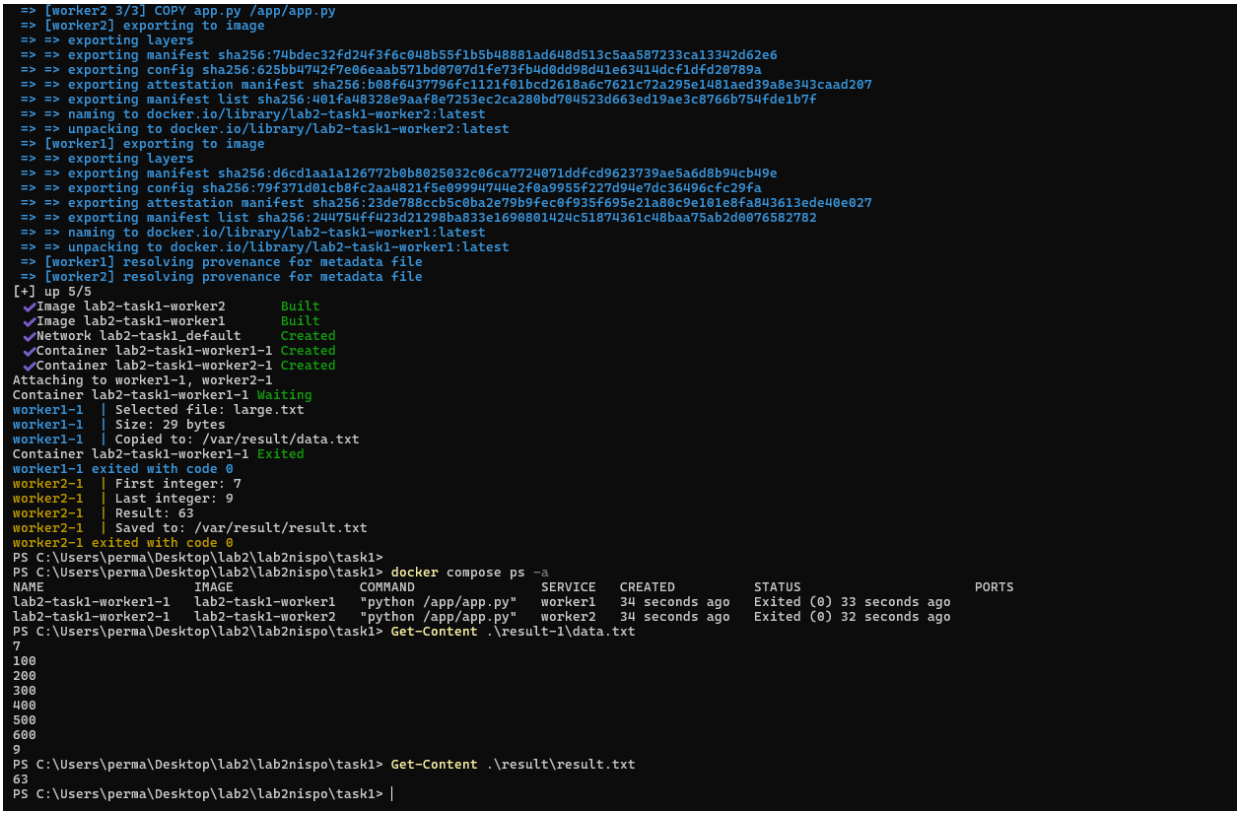


Рисунок 13. Завершение сборки и работы контейнеров с результатом 63

5 Сохранение задания 1 в Git

Я добавил .gitignore, который исключает виртуальное окружение, временные файлы Python и результаты вычислений. Пустые каталоги результатов сохраняются благодаря .gitkeep. В каталогах обоих worker размещены .dockerignore для соответствующих контекстов сборки; дополнительный файл имеется в task1 по требованию задания.

Команда git check-ignore подтвердила исключение result-1/data.txt и result/result.txt. Исходные файлы с числами остались в репозитории. После оформления README я сохранил 15 файлов коммитом f9e0af1 в ветке task1. Проверка git status показала чистое рабочее дерево.

```
PS C:\Users\perma\Desktop\lab2\lab2nispo\task1> git commit -m "Complete task1: file processing with Docker Compose"
[task1 f9e0af1] Complete task1: file processing with Docker Compose
 15 files changed, 273 insertions(+)
 create mode 100644 task1/.dockerignore
 create mode 100644 task1/.gitignore
 create mode 100644 task1/README.md
 create mode 100644 task1/compose.yaml
 create mode 100644 task1/data/large.txt
 create mode 100644 task1/data/medium.txt
 create mode 100644 task1/data/small.txt
 create mode 100644 task1/result-1/.gitkeep
 create mode 100644 task1/result/.gitkeep
 create mode 100644 task1/worker1/.dockerignore
 create mode 100644 task1/worker1/Dockerfile
 create mode 100644 task1/worker1/app.py
 create mode 100644 task1/worker2/.dockerignore
 create mode 100644 task1/worker2/Dockerfile
 create mode 100644 task1/worker2/app.py
PS C:\Users\perma\Desktop\lab2\lab2nispo\task1> git status
On branch task1
nothing to commit, working tree clean
PS C:\Users\perma\Desktop\lab2\lab2nispo\task1> git log --oneline --decorate -2
f9e0af1 (HEAD -> task1) Complete task1: file processing with Docker Compose
be7d8e5 (main) Initialize lab 2
PS C:\Users\perma\Desktop\lab2\lab2nispo\task1>
```

Рисунок 19. Коммит задания 1 и состояние ветки task1

Публикация отложена до завершения локальной работы. Ветка task2 была создана от исходной main, пока task1 ещё не слита. Перед окончательной сдачей необходимо оформить Pull Request task1 → main, обновить task2 изменениями main, затем оформить Pull Request task2 → main. Обе ветки следует сохранить.

6 Упражнение 2 2 и запуск RabbitMQ

Для упражнения 2.2 я создал task2-2 с каталогами lesson1, lesson2 и lesson3 и отдельным окружением .venv. По требованию для группы 13 клиентские программы написаны на Python и запускаются на Windows, а в контейнере работает только RabbitMQ.

В compose.yaml я использовал rabbitmq:4-management. Во время выполнения была загружена версия RabbitMQ 4.3.6. Порт 5672 доступен для AMQP, порт 15672 — для интерфейса управления. Оба порта опубликованы на 127.0.0.1. Учебные данные доступа: labuser / labpass. Данные брокера размещены в именованном томе rabbitmq-data.

```
docker compose config -q
docker compose up -d --wait
docker compose ps
```

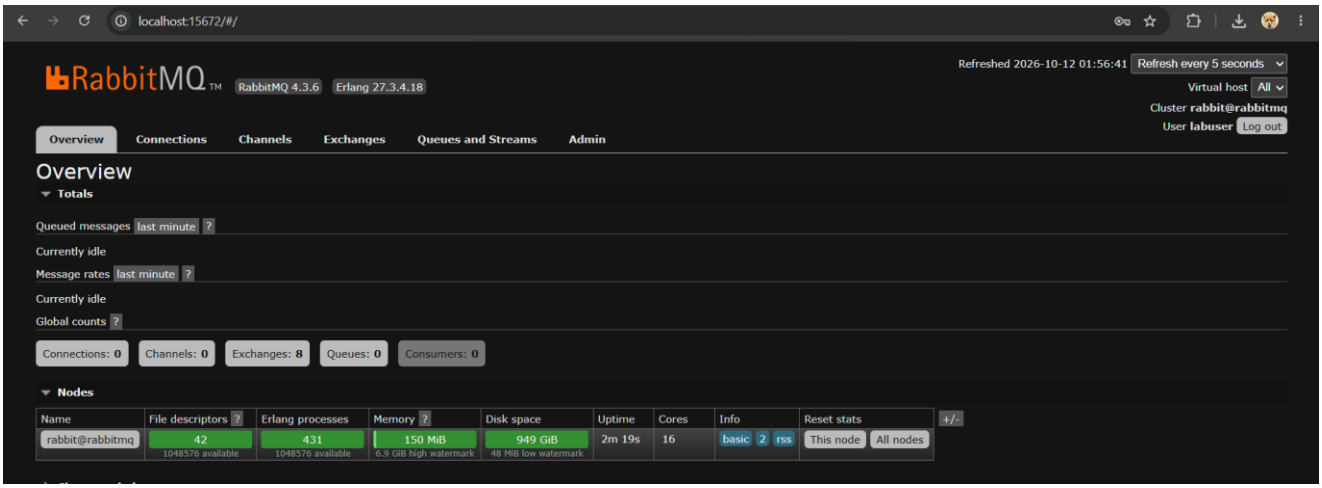


Рисунок 23. Вход в интерфейс RabbitMQ под пользователем labuser

Я установил Pika 1.4.4 и сохранил pika==1.4.4 в requirements.txt. Проверка pip check не обнаружила нарушенных зависимостей. Тестовое AMQP-подключение вывело Connection to RabbitMQ: OK. Проверка healthcheck использует rabbitmq-diagnostics -q ping.

7 Урок 1 Hello World

В send.py я создаю соединение с брокером, объявляю очередь hello и публикую Hello World! через обменник по умолчанию с routing_key=hello. В receive.py я подписываюсь на ту же очередь и вывожу полученное сообщение. Параметры объявления очереди совпадают: durable=True и тип quorum.

Я запустил получателя в одном окне PowerShell, а отправителя — в другом. Отправитель завершился после отправки, получатель продолжил ожидать новые сообщения. В этом учебном примере используется auto_ack=True.



Рисунок 27. Отправка Hello World



Рисунок 26. Получение Hello World

Затем я остановил получателя, отправил ещё одно сообщение и увидел Ready=1, Unacked=0 в очереди hello. После повторного запуска получателя сообщение было доставлено. Эта проверка подтверждает ожидание получателя в очереди, но не проверяет сохранение сообщения первого урока при перезапуске брокера.

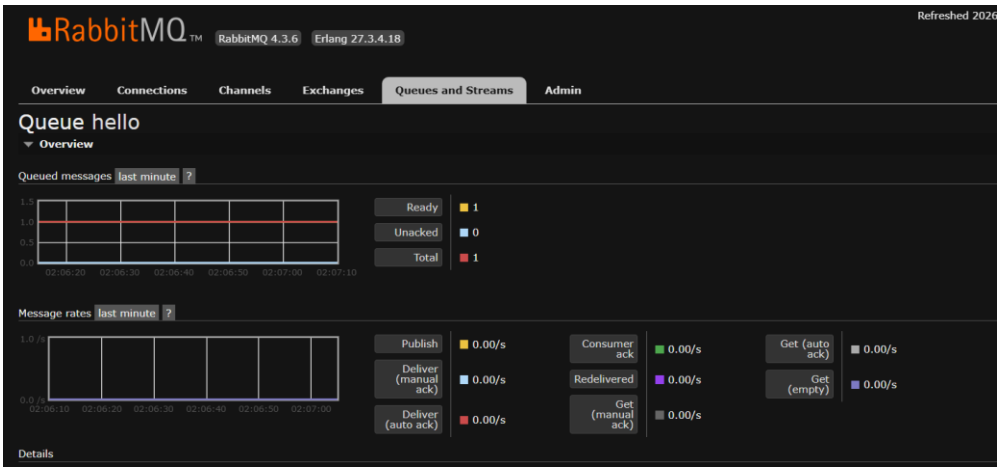


Рисунок 30. Сообщение ожидает получателя в очереди hello

8 Урок 2 Распределение задач

В new_task.py я публикую задачи в task_queue. Сообщение задаётся аргументом командной строки, каждая точка моделирует секунду обработки. В worker.py подтверждение basic_ack отправляется после обработки. Параметр prefetch_count=1 ограничивает обработчика одной неподтверждённой задачей.

Я запустил два обработчика. На скриншотах процесс 17604 выполнил задачи 1 и 4, а процесс 22060 — задачи 2, 3 и 5. Все пять задач завершились выводом Done. Распределение зависит от скорости обработки, поэтому оно не обязано быть строго по очереди.

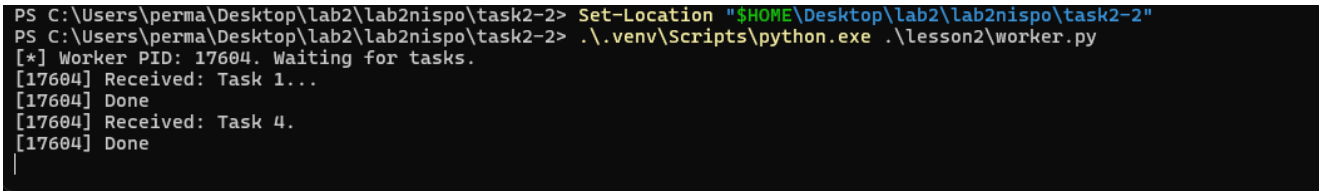


Рисунок 35. Первый обработчик выполнил задачи 1 и 4

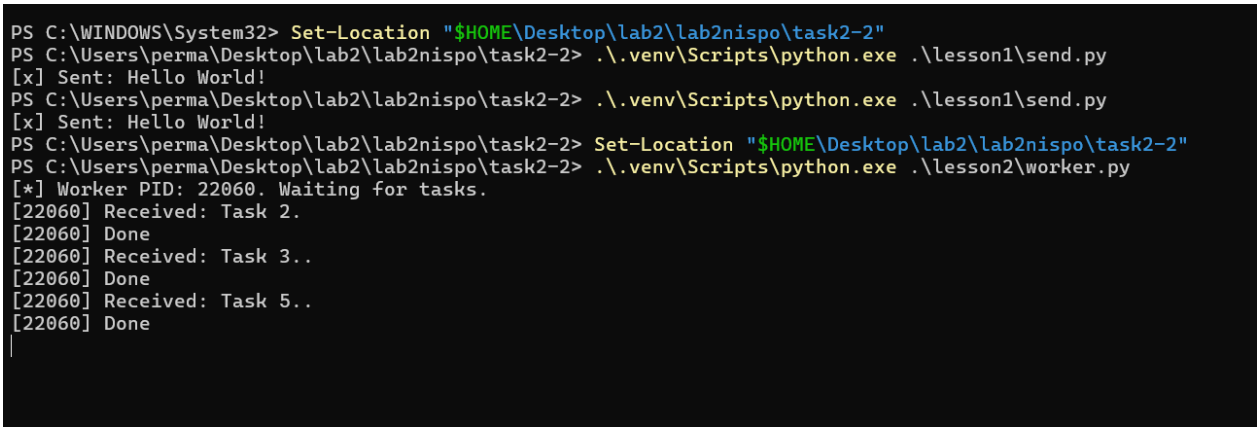


Рисунок 36. Второй обработчик выполнил задачи 2 3 и 5

Очередь объявлена durable и имеет тип quorum. Сообщения публикуются с delivery_mode=2. Отправитель включает confirm_delivery и использует mandatory=True. Подтверждение публикации относится к приёму сообщения брокером, а basic_ack — к завершению обработки потребителем.

9 Повторная доставка незавершённой задачи

Я отправил Retry-test с 30 точками и остановил обработчика после строки Received, до появления Done. Поскольку подтверждение не было отправлено, сообщение вернулось в очередь. В интерфейсе видно Ready=1 и Unacked=0.

```
[17604] Received: Retry-test..............................
Worker stopped.
PS C:\Users\perma\Desktop\lab2\lab2nispo\task2-2>
```

Рисунок 38. Остановка обработчика до подтверждения задачи

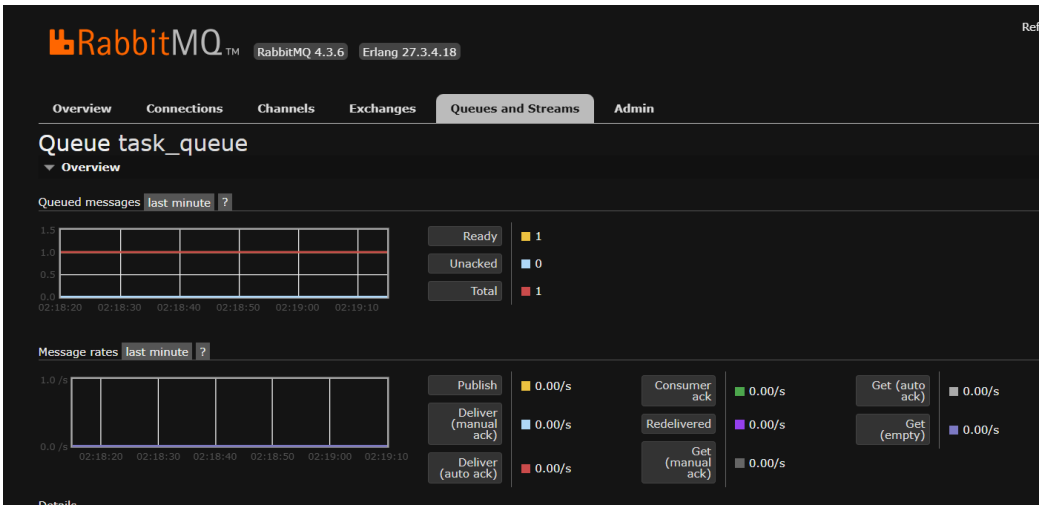


Рисунок 39. Незавершённая задача снова доступна в очереди

После запуска другого обработчика с PID 21468 та же задача была получена повторно и завершилась выводом Done. Такой подход допускает повторную обработку при сбоях; для реальных операций потребуются идемпотентность или контроль дубликатов.

```
PS C:\Users\perma\Desktop\lab2\lab2nispo\task2-2> .\.venv\Scripts\python.exe .\lesson2\worker.py
[*] Worker PID: 21468. Waiting for tasks.
[21468] Received: Retry-test..............................
[21468] Done
```

Рисунок 40. Другой обработчик завершил Retry-test

10 Проверка перезапуска брокера

При остановленных обработчиках я отправил Restart-test..., перезапустил RabbitMQ командой docker compose restart rabbitmq и дождался готовности через docker compose up -d --wait. Контейнер снова получил статус healthy.

```
PS C:\Users\perma\Desktop\lab2\lab2nispo\task2-2> .\.venv\Scripts\python.exe .\lesson2\new_task.py "Restart-test..."
[x] Sent: Restart-test...
PS C:\Users\perma\Desktop\lab2\lab2nispo\task2-2>
```

Рисунок 42. Отправка задачи Restart-test

```
PS C:\Users\perma\Desktop\lab2\lab2nispo\task2-2> docker compose restart rabbitmq
[+] restart 0/1
 - Container lab2-task2-rabbitmq-1 Restarting                                                                    2.0s
PS C:\Users\perma\Desktop\lab2\lab2nispo\task2-2> docker compose up -d --wait
[+] up 1/1
 ✔ Container lab2-task2-rabbitmq-1 Healthy                                                                       5.5s
PS C:\Users\perma\Desktop\lab2\lab2nispo\task2-2> docker compose ps
NAME                   IMAGE                  COMMAND                  SERVICE    CREATED          STATUS
      PORTS
lab2-task2-rabbitmq-1   rabbitmq:4-management   "docker-entrypoint.s…"   rabbitmq   29 minutes ago   Up 8 seconds (healt
hy)   127.0.0.1:5672->5672/tcp, 127.0.0.1:15672->15672/tcp
PS C:\Users\perma\Desktop\lab2\lab2nispo\task2-2>
```

Рисунок 41. Перезапуск RabbitMQ и проверка готовности

После запуска worker.py процесс 22416 получил Restart-test... и вывел Done. Это проверка обычного перезапуска одного брокера с сохранённым томом. Отказ диска, удаление тома или отказ нескольких узлов кластера в работе не моделировались. Порядок вставки кадров в исходный Word отличается от логического порядка теста; здесь сначала показана отправка задачи.

```
PS C:\Users\perma\Desktop\lab2\lab2nispo\task2-2> .\.venv\Scripts\python.exe .\lesson2\worker.py
[*] Worker PID: 22416. Waiting for tasks.
[22416] Received: Restart-test...
[22416] Done
```

Рисунок 43. Получение и завершение задачи после перезапуска

11 Урок 3 Рассылка подписчикам

В emit_log.py я объявляю обменник logs типа fanout. Каждый экземпляр receive_logs.py создаёт отдельную эксклюзивную очередь с автоматически выданным именем и связывает её с logs. Публикация выполняется после запуска обоих подписчиков.

Сообщение Lab2 group13: broadcast test получили оба процесса: 9388 и 21912. В отличие от общей рабочей очереди второго урока здесь каждый подписчик получает свою копию сообщения через собственную очередь.

```
[*] Waiting for logs. Press Ctrl+C to exit.
[9388] Received: Lab2 group13: broadcast test
```

Рисунок 48. Получение сообщения подписчиком 9388

```
[*] Waiting for logs. Press Ctrl+C to exit.
[21912] Received: Lab2 group13: broadcast test
```

Рисунок 49. Получение сообщения подписчиком 21912

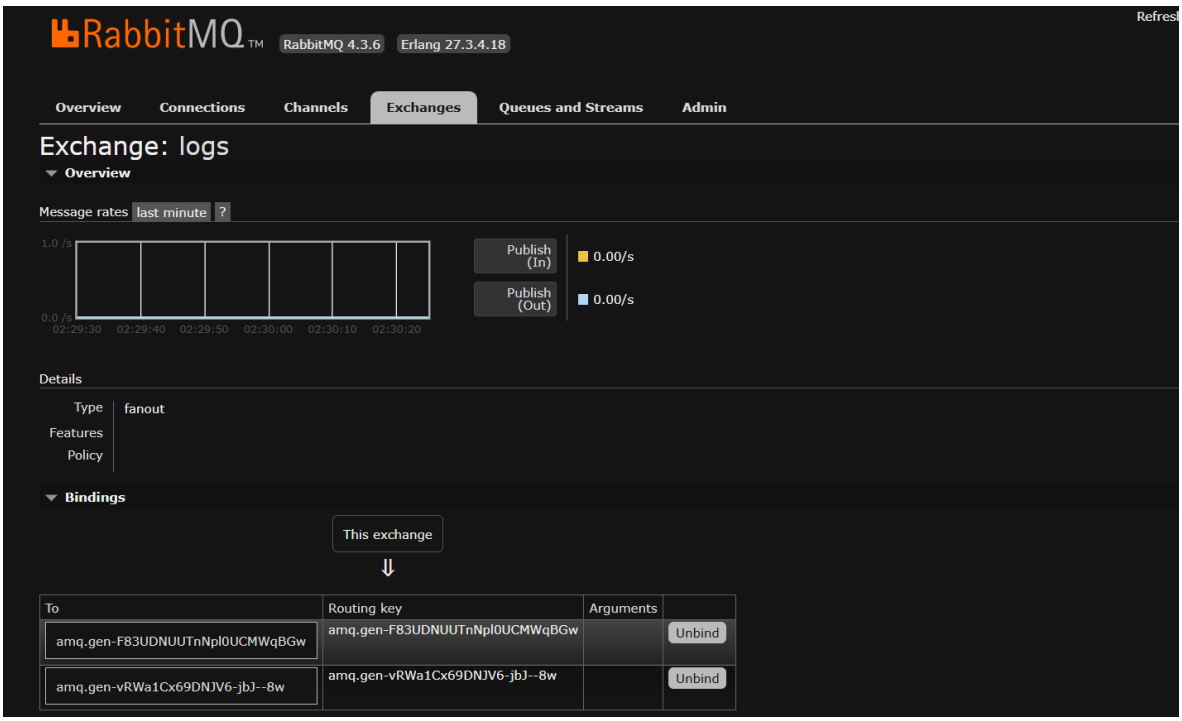


Рисунок 50. Обменник fanout и связи с двумя очередями

12 Отключение и повторное подключение подписчика

Я отключил одного подписчика и отправил Only active subscriber. Сообщение получил оставшийся процесс 21912. Затем я снова запустил отключённого подписчика: процесс 15248 получил новую очередь. Старое сообщение в неё не поступило.

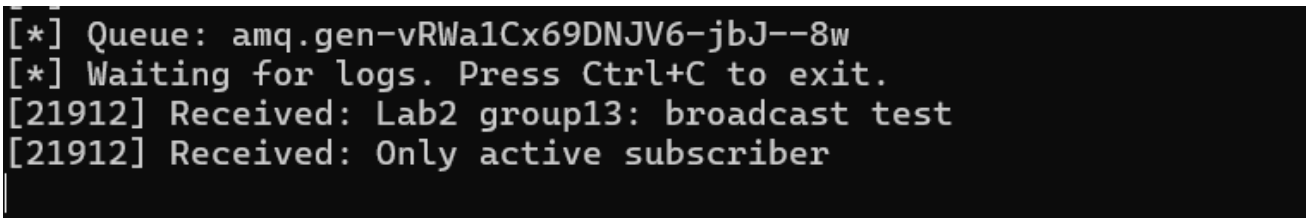


Рисунок 51. Сообщение получает оставшийся активный подписчик

После публикации Both subscribers online в материалах зафиксировано получение сообщения повторно подключённым процессом 15248. Отдельного кадра получения именно этой последней фразы вторым процессом нет. Рассылка двум подписчикам ранее подтверждена обоими кадрами для broadcast test.



Рисунок 55. Новая очередь и получение нового сообщения после подключения

Эксклюзивная очередь удаляется при закрытии соединения. Поэтому этот пример рассылки не обеспечивает хранение истории для отсутствующего подписчика. Для такого требования нужна заранее созданная долговечная очередь и другая политика её жизненного цикла.

Результат практической части

Я сравнил обмен через файлы, рабочую очередь и рассылку. В задании 1 два контейнера последовательно получили итог 63. RabbitMQ позволил отделить отправителя от получателя, распределить задачи и повторно доставить неподтверждённую задачу. Практическая инфраструктура Kafka не создавалась: для группы 13 по ней требуются ответы на контрольные вопросы.

13 Контрольные вопросы

1 Что такое обмен сообщениями и какие проблемы он решает

Обмен сообщениями — передача данных между компонентами через сообщения. Он уменьшает зависимость отправителя от доступности и скорости получателя, позволяет буферизовать нагрузку и выполнять работу асинхронно. Для надёжной обработки отдельно проектируют подтверждения, повторы и защиту от дубликатов.

2 Что такое брокер сообщений и какова его роль

Брокер — посредник между издателями и потребителями. Он принимает публикации, маршрутизирует их, хранит сообщения в очередях и доставляет потребителям. Он также управляет доступом и состоянием доставки. Бизнес-логику обработки выполняют клиентские приложения.

3 Какие основные шаблоны обмена сообщениями существуют

Point-to-point передаёт задачу одному потребителю; competing consumers распределяет задачи общей очереди между обработчиками; publish/subscribe доставляет событие нескольким подписчикам через отдельные подписки или очереди; request/reply связывает запрос и ответ идентификатором корреляции. Потоковая модель хранит последовательность событий для независимого чтения.

4 Дайте определение RabbitMQ

RabbitMQ — брокер сообщений с поддержкой очередей, маршрутизации и нескольких протоколов. В этой работе он принимает сообщения Python-программ по AMQP 0-9-1 и доставляет их обработчикам либо очередям подписчиков.

5 Какой протокол лежит в основе RabbitMQ

Для традиционной модели exchanges и queues и для использованного клиента Pika применяется AMQP 0-9-1. RabbitMQ также поддерживает AMQP 1.0 и другие протоколы, в том числе MQTT и STOMP с соответствующими возможностями сервера. Нельзя считать эти версии AMQP одним и тем же протоколом.

Контрольные вопросы продолжение

6 Куда записываются сообщения от приложения издателя

В модели AMQP 0-9-1 издатель направляет сообщение обменнику, который маршрутизирует его в подходящие очереди. Именно очереди хранят ожидающие доставки сообщения; обменник сам не является очередью. При отсутствии подходящего маршрута хранение не происходит автоматически.

7 Что такое exchange

Exchange — точка маршрутизации сообщений. Она выбирает очереди по своему типу, bindings и атрибутам публикации. Например, fanout направляет копию во все связанные очереди, а direct сопоставляет ключ маршрутизации с ключом привязки. [2]

8 Что такое binding

Binding — связь обменника с очередью или другим обменником, задающая возможность маршрутизации. В зависимости от типа exchange привязка может включать ключ или аргументы. В уроке 3 queue_bind связывает каждую временную очередь с logs. [2]

9 Что такое routing key

Routing key — строка, которую издатель передаёт при публикации. Direct использует точное совпадение, topic — шаблоны, а fanout её игнорирует. У обменника по умолчанию имя очереди служит ключом маршрутизации. [2]

10 Как сообщение проходит от издателя до потребителя

Клиент открывает соединение и канал, объявляет необходимую топологию, затем публикует сообщение в exchange. Обменник выбирает связанные очереди. Очередь хранит сообщение до доставки подходящему потребителю. При ручном подтверждении потребитель после обработки отправляет ack, позволяя брокеру удалить доставленное сообщение. [2, 3]

Контрольные вопросы продолжение

11 Какой обменник выбрать для распараллеливания обработки

Достаточно обменника по умолчанию или direct, маршрутизирующего задачи в одну общую очередь. Несколько потребителей подписываются на неё. Распределение выполняет очередь между потребителями; само по себе создание fanout с отдельной очередью каждому обработчику привело бы к копированию задач. [1, 2]

12 Какие основные типы exchanges существуют

Direct использует точное совпадение ключа; fanout рассылает во все связанные очереди; topic сопоставляет routing key с шаблоном, где * означает одно слово, а # — ноль или больше слов; headers сопоставляет заголовки по условиям привязки. Обменник по умолчанию — специальный direct с именем пустой строки. [2]

13 Куда издатель обычно отправляет сообщение

Издатель публикует сообщение в exchange, задавая routing key и свойства. Даже кажущаяся прямой отправка в очередь использует обменник по умолчанию. Например, exchange="" и routing_key="hello" направляют сообщение в очередь hello. [1]

14 Как предотвратить потребление устаревших данных

Настроить TTL сообщений на очереди или срок expiration при публикации. Просроченные сообщения не должны доставляться как обычные. Если актуальность критична во время обработки, потребитель дополнительно проверяет временную метку или срок действия: TTL очереди не отменяет уже начатую работу. Просроченные сообщения можно направлять в DLX. [4]

15 По какому принципу устроены очереди — FIFO или LIFO

Базовая дисциплина — FIFO. Однако наблюдаемый порядок обработки может изменяться из-за нескольких потребителей, повторной доставки и приоритетов. Порядок завершения задач также зависит от длительности обработки. FIFO не означает глобального порядка для всей распределённой системы. [5]

Контрольные вопросы продолжение

16 Для чего предназначен Server Broker Node

Узел RabbitMQ принимает клиентские соединения, обслуживает каналы, обменники и очереди, маршрутизирует сообщения и управляет доставкой. Узлы могут объединяться в кластер. Репликация данных зависит от типа очереди и конфигурации; наличие кластера само по себе не делает каждую очередь реплицированной. [5]

17 Что такое virtual host

Vhost — логическое пространство имён и прав внутри RabbitMQ. В разных vhost могут существовать одноимённые очереди и обменники. Доступ выдаётся пользователям для конкретного vhost. Это логическая изоляция, а не отдельный процесс или гарантированное разделение аппаратных ресурсов. В работе используется vhost /. [6]

18 Что такое ack nack reject

Ack подтверждает успешную обработку. Nack и reject отклоняют доставку: с requeue=true сообщение возвращается в очередь, с false может попасть в DLX либо быть отброшено. Nack поддерживает групповое отклонение, reject — одного сообщения. Publisher confirms отдельно подтверждают приём публикации брокером. Эти механизмы не дают автоматической гарантии exactly-once для бизнес-операции. [3]

19 Чем отличаются durable и transient очереди и сообщения

Durable-очередь сохраняет своё определение после перезапуска; transient-очередь — нет. Для сообщений отдельно задаётся persistent, например delivery_mode=2. Это разные свойства: долговечность очереди не заменяет настройку публикации. Для надёжной отправки применяют publisher confirms и подходящий тип очереди. В работе task_queue — quorum; данные хранятся в томе. [3, 5]

20 Что такое Dead Letter Exchange

DLX — обменник, куда очередь перенаправляет сообщения при определённых событиях: отказ без requeue, истечение TTL, превышение ограничения длины или лимита доставок для quorum-очереди. Далее сообщение маршрутизируется обычными bindings. DLX используют для разбора ошибок, карантина и контролируемых повторов; он не является отдельным видом очереди. [7]

Контрольные вопросы продолжение

21 Что такое Priority Queue

Приоритетная очередь учитывает свойство priority при выборе следующего сообщения. Возможности настройки зависят от типа очереди и версии RabbitMQ: нельзя переносить настройки classic-очереди на quorum без проверки. Высокий приоритет не прерывает уже доставленную обработчику задачу; большое значение prefetch может уменьшить заметность приоритетов. В лабораторной приоритеты не настраивались. [8]

22 Что такое Apache Kafka и какие задачи она решает

Kafka — распределённая платформа потоков событий. Она хранит записи в разделённых на partition топиках и позволяет независимым приложениям читать историю. Её используют для журналов событий, интеграции данных и потоковой аналитики. Чтение записи обычно не удаляет её: срок хранения определяется политикой retention или compaction. [9]

23 Как устроена Kafka и что такое Broker Topic Partition

Broker — сервер Kafka; topic — именованный поток записей; partition — упорядоченный журнал внутри топика. Каждая запись имеет offset в своей partition. Разбиение позволяет распределять хранение и обработку. Порядок гарантируется в пределах раздела, а не автоматически для всего топика. [9, 10]

24 Что такое Consumer Group

Consumer group объединяет потребителей для совместного чтения. При обычной групповой подписке каждую partition в данный момент читает один участник группы. Добавление участников увеличивает параллелизм до числа разделов. Независимые группы читают тот же топик со своими offsets. При изменении состава происходит перераспределение разделов. [10]

25 Что такое Replica Leader и Follower

Replica — копия partition. Leader обслуживает запись в раздел, followers копируют журнал. При отказе лидера возможен выбор нового из допустимых реплик. Надёжность зависит от числа реплик, состояния ISR и настроек подтверждений. Репликация повышает доступность, но не заменяет резервное копирование и корректную конфигурацию. [10]

Контрольные вопросы продолжение

26 Для чего нужны ZooKeeper или KRaft

Контроллеры управляют метаданными кластера, составом узлов и лидерами разделов. Исторически Kafka использовала ZooKeeper. KRaft хранит метаданные в журнале с консенсусом Raft и выделяет роли broker и controller. В Kafka 4.x используется KRaft; ZooKeeper относится к прежним версиям. [11]

27 Что такое Kafka Streams

Kafka Streams — библиотека Java для обработки потоков Kafka внутри приложения. Она предоставляет операции преобразования, группировки, соединения, окон и управления состоянием. Обычный Consumer API даёт чтение записей, а логику состояния, времени и обработки разработчик организует сам. Kafka Streams не требует отдельного сервера обработки как самостоятельного продукта. [12]

28 Каковы архитектурные различия RabbitMQ и Kafka

В обычных очередях RabbitMQ обменник маршрутизирует сообщения, а подтверждённые доставки удаляются из очереди. В Kafka записи остаются в partition по политике хранения, а прогресс потребителей задаётся offsets. RabbitMQ удобен для адресуемых задач и маршрутизации, Kafka — для повторного чтения журналов. У RabbitMQ также есть streams, поэтому это сравнение относится прежде всего к классической модели очередей. [5, 9, 10]

29 Когда выбирать RabbitMQ а когда Kafka

RabbitMQ подходит для фоновых заданий, уведомлений и сложной маршрутизации сообщений между сервисами. Kafka подходит для длительного журнала событий, интеграции данных, аналитики и нескольких независимых читателей. Выбор зависит от требований к задержке, истории, повторному чтению и эксплуатации, а не только от названия технологии. [9, 10]

30 Можно ли использовать RabbitMQ и Kafka вместе

Да. Например, Kafka хранит события заказов для аналитики и повторной обработки, а RabbitMQ распределяет конкретные задачи отправки писем между рабочими процессами. При соединении систем нужно определить идентификаторы событий, повторы и обработку дубликатов. Такая архитектура — пример проектного решения, а не часть выполненной практической работы.

14 Источники

Методические указания к лабораторной работе 2, БГУ, 2026–2027. Страницы 4–10 и 13–14. Приложенный PDF.

1 RabbitMQ Tutorials Python
https://www.rabbitmq.com/tutorials/tutorial-one-python

1 RabbitMQ Work Queues
https://www.rabbitmq.com/tutorials/tutorial-two-python

1 RabbitMQ Publish Subscribe
https://www.rabbitmq.com/tutorials/tutorial-three-python

2 RabbitMQ Exchanges
https://www.rabbitmq.com/docs/exchanges

3 Consumer Acknowledgements and Publisher Confirms
https://www.rabbitmq.com/docs/confirms

4 Time To Live and Expiration
https://www.rabbitmq.com/docs/ttl

5 RabbitMQ Queues
https://www.rabbitmq.com/docs/queues

6 Virtual Hosts
https://www.rabbitmq.com/docs/vhosts

7 Dead Letter Exchanges
https://www.rabbitmq.com/docs/dlx

8 Priority Support in Queues
https://www.rabbitmq.com/docs/priority

9 Apache Kafka Introduction
https://kafka.apache.org/41/getting-started/introduction/

10 Apache Kafka Design
https://kafka.apache.org/41/design/design/

11 Apache Kafka KRaft
https://kafka.apache.org/41/operations/kraft/

12 Kafka Streams
https://kafka.apache.org/41/streams/

Приложение А Листинг 1

task1/worker1/app.py

```
import argparse
import shutil
import sys
from pathlib import Path


def main():
    parser = argparse.ArgumentParser(
        description="Select the largest input file by size in bytes."
    )
    parser.add_argument("--input-dir", type=Path, default=Path("/var/data"))
    parser.add_argument("--output-dir", type=Path, default=Path("/var/result"))
    args = parser.parse_args()
    try:
        if not args.input_dir.is_dir():
            raise ValueError(f"Input directory not found: {args.input_dir}")
        files = [
            path for path in args.input_dir.iterdir()
            if path.is_file() and path.name != ".gitkeep"
        ]
        if not files:
            raise ValueError("Input directory contains no data files.")
        selected = sorted(
            files,
            key=lambda path: (-path.stat().st_size, path.name),
        )[0]
        args.output_dir.mkdir(parents=True, exist_ok=True)
        destination = args.output_dir / "data.txt"
        shutil.copyfile(selected, destination)
        print(f"Selected file: {selected.name}")
        print(f"Size: {selected.stat().st_size} bytes")
        print(f"Copied to: {destination}")
        return 0
    except (OSError, ValueError) as error:
        print(f"Error: {error}", file=sys.stderr)
        return 1


if __name__ == "__main__":
    sys.exit(main())
```

Приложение А Листинг 2

task1/worker2/app.py

```
import argparse
import sys
from pathlib import Path


def main():
    parser = argparse.ArgumentParser(
        description="Multiply the first and last integers in data.txt."
    )
    parser.add_argument("--input-dir", type=Path, default=Path("/var/data"))
    parser.add_argument("--output-dir", type=Path, default=Path("/var/result"))
    args = parser.parse_args()
    try:
        source = args.input_dir / "data.txt"
        tokens = source.read_text(encoding="utf-8-sig").split()
        if not tokens:
            raise ValueError("Input file contains no integers.")
        numbers = []
        for position, token in enumerate(tokens, start=1):
            try:
                numbers.append(int(token))
            except ValueError:
                raise ValueError(
                    f"Invalid integer at position {position}: {token!r}"
                ) from None
        result = numbers[0] * numbers[-1]
        args.output_dir.mkdir(parents=True, exist_ok=True)
        destination = args.output_dir / "result.txt"
        destination.write_text(f"{result}\n", encoding="utf-8")
        print(f"First integer: {numbers[0]}")
        print(f"Last integer: {numbers[-1]}")
        print(f"Result: {result}")
        print(f"Saved to: {destination}")
        return 0
    except (OSError, UnicodeError, ValueError) as error:
        print(f"Error: {error}", file=sys.stderr)
        return 1


if __name__ == "__main__":
    sys.exit(main())
```

Приложение А Листинг 3

task1/worker1/Dockerfile

Для worker2 используется такой же Dockerfile.

```
# Base image with Python 3.14.
FROM python:3.14-slim

# Disable bytecode files and enable immediate log output.
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# Copy the application into the image.
WORKDIR /app
COPY app.py /app/app.py

# Run the worker using its default container paths.
CMD ["python", "/app/app.py"]
```

Приложение А Листинг 4

task1/compose.yaml

```
name: lab2-task1
services:
  worker1:
    build:
      context: ./worker1
    volumes:
      - type: bind
        source: ./data
        target: /var/data
        read_only: true
      - type: bind
        source: ./result-1
        target: /var/result
  worker2:
    build:
      context: ./worker2
    depends_on:
      worker1:
        condition: service_completed_successfully
    volumes:
      - type: bind
        source: ./result-1
        target: /var/data
        read_only: true
      - type: bind
        source: ./result
        target: /var/result
```

Приложение А Листинг 5

task2-2/compose.yaml

```
name: lab2-task2
services:
  rabbitmq:
    image: rabbitmq:4-management
    hostname: rabbitmq
    environment:
      RABBITMQ_DEFAULT_USER: labuser
      RABBITMQ_DEFAULT_PASS: labpass
    ports:
      - "127.0.0.1:5672:5672"
      - "127.0.0.1:15672:15672"
    volumes:
      - rabbitmq-data:/var/lib/rabbitmq
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
      timeout: 5s
      retries: 12
      start_period: 30s
volumes:
  rabbitmq-data:
```

Приложение А Листинг 6

task2-2/lesson1/send.py

```
import pika

parameters = pika.ConnectionParameters(
    host="127.0.0.1",
    credentials=pika.PlainCredentials("labuser", "labpass"),
)

with pika.BlockingConnection(parameters) as connection:
    channel = connection.channel()
    channel.queue_declare(
        queue="hello",
        durable=True,
        arguments={"x-queue-type": "quorum"},
    )
    channel.basic_publish(
        exchange="",
        routing_key="hello",
        body="Hello World!",
    )
    print("[x] Sent: Hello World!")
```

Приложение А Листинг 7

task2-2/lesson1/receive.py

```
import pika

parameters = pika.ConnectionParameters(
    host="127.0.0.1",
    credentials=pika.PlainCredentials("labuser", "labpass"),
)

def callback(channel, method, properties, body):
    print(f"[x] Received: {body.decode('utf-8')}", flush=True)


def main():
    with pika.BlockingConnection(parameters) as connection:
        channel = connection.channel()
        channel.queue_declare(
            queue="hello",
            durable=True,
            arguments={"x-queue-type": "quorum"},
        )
        channel.basic_consume(
            queue="hello",
            on_message_callback=callback,
            auto_ack=True,
        )
        print("[*] Waiting for messages. Press Ctrl+C to exit.", flush=True)
        try:
            channel.start_consuming()
        except KeyboardInterrupt:
            channel.stop_consuming()
            print("Receiver stopped.")


if __name__ == "__main__":
    main()
```

Приложение А Листинг 8

task2-2/lesson2/new_task.py

```
import sys
import pika

parameters = pika.ConnectionParameters(
    host="127.0.0.1",
    credentials=pika.PlainCredentials("labuser", "labpass"),
)

message = " ".join(sys.argv[1:]) or "Hello World!"

with pika.BlockingConnection(parameters) as connection:
    channel = connection.channel()
    channel.queue_declare(
        queue="task_queue",
        durable=True,
        arguments={"x-queue-type": "quorum"},
    )
    channel.confirm_delivery()
    channel.basic_publish(
        exchange="",
        routing_key="task_queue",
        body=message.encode("utf-8"),
        properties=pika.BasicProperties(delivery_mode=2),
        mandatory=True,
    )
    print(f"[x] Sent: {message}")
```

Приложение А Листинг 9

task2-2/lesson2/worker.py

```
import os
import time
import pika

parameters = pika.ConnectionParameters(
    host="127.0.0.1",
    credentials=pika.PlainCredentials("labuser", "labpass"),
)

def callback(channel, method, properties, body):
    message = body.decode("utf-8")
    print(f"[{os.getpid()}] Received: {message}", flush=True)
    time.sleep(message.count("."))
    channel.basic_ack(delivery_tag=method.delivery_tag)
    print(f"[{os.getpid()}] Done", flush=True)


def main():
    with pika.BlockingConnection(parameters) as connection:
        channel = connection.channel()
        channel.queue_declare(
            queue="task_queue",
            durable=True,
            arguments={"x-queue-type": "quorum"},
        )
        channel.basic_qos(prefetch_count=1)
        channel.basic_consume(
            queue="task_queue",
            on_message_callback=callback,
            auto_ack=False,
        )
        print(f"[*] Worker PID: {os.getpid()}. Waiting for tasks.", flush=True)
        try:
            channel.start_consuming()
        except KeyboardInterrupt:
            channel.stop_consuming()
            print("Worker stopped.")


if __name__ == "__main__":
    main()
```

Приложение А Листинг 10

task2-2/lesson3/emit_log.py

```
import sys
import pika

parameters = pika.ConnectionParameters(
    host="127.0.0.1",
    credentials=pika.PlainCredentials("labuser", "labpass"),
)

message = " ".join(sys.argv[1:]) or "Hello subscribers!"

with pika.BlockingConnection(parameters) as connection:
    channel = connection.channel()
    channel.exchange_declare(exchange="logs", exchange_type="fanout")
    channel.basic_publish(
        exchange="logs",
        routing_key="",
        body=message.encode("utf-8"),
    )
    print(f"[x] Sent: {message}")
```

Приложение А Листинг 11

task2-2/lesson3/receive_logs.py

```
import os
import pika

parameters = pika.ConnectionParameters(
    host="127.0.0.1",
    credentials=pika.PlainCredentials("labuser", "labpass"),
)

def callback(channel, method, properties, body):
    print(f"[{os.getpid()}] Received: {body.decode('utf-8')}", flush=True)


def main():
    with pika.BlockingConnection(parameters) as connection:
        channel = connection.channel()
        channel.exchange_declare(exchange="logs", exchange_type="fanout")
        result = channel.queue_declare(queue="", exclusive=True)
        queue_name = result.method.queue
        channel.queue_bind(exchange="logs", queue=queue_name)
        channel.basic_consume(
            queue=queue_name,
            on_message_callback=callback,
            auto_ack=True,
        )
        print(f"[*] Subscriber PID: {os.getpid()}", flush=True)
        print(f"[*] Queue: {queue_name}", flush=True)
        print("[*] Waiting for logs. Press Ctrl+C to exit.", flush=True)
        try:
            channel.start_consuming()
        except KeyboardInterrupt:
            channel.stop_consuming()
            print("Subscriber stopped.")


if __name__ == "__main__":
    main()
```

Приложение Б Зависимости и исключения

task2-2/requirements.txt

```
pika==1.4.4
```

task1/.gitignore

```
__pycache__/
*.py[cod]
.venv/
venv/
/result-1/*
!/result-1/.gitkeep
/result/*
!/result/.gitkeep
```

task2-2/.gitignore

```
.venv/
__pycache__/
*.py[cod]
.env
```

В task1 и каталогах worker1, worker2 файлы .dockerignore исключают __pycache__/, *.py[cod], .venv/ и .git/. В task2-2 дополнительно исключён .env. В упражнении 2.2 собственная сборка образа не выполняется, поэтому этот .dockerignore при текущем запуске не используется.

Пустые файлы result-1/.gitkeep и result/.gitkeep сохраняют каталоги в Git. В каталоге data лежат исходные числовые файлы из раздела 2.